Spotify

Team Members:

Andrew Rauscher, Kyle Dodson, Abbi Parks, Henry Chen, Joseph Valeski

April 28, 2026

ISA 345

Section C

Project Overview

In a world where data tracking is gaining importance, the demand for identifying personal trends and creating tailor-made data is higher than ever. In this project our team will be overseeing and coordinating the various business processes of Spotify.

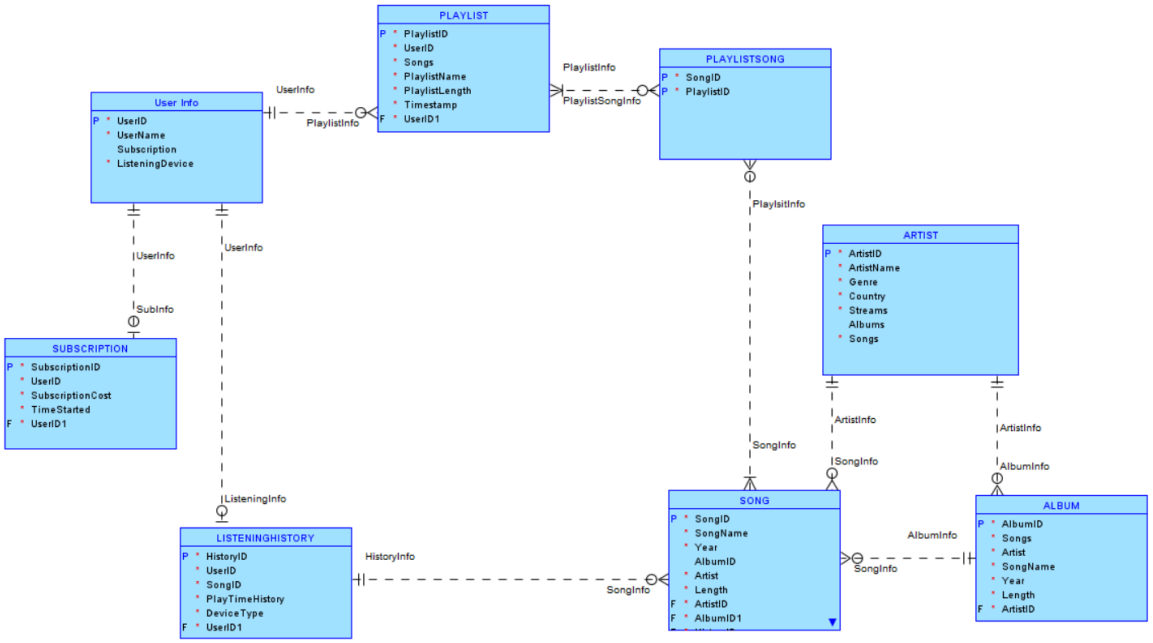
To start, we need to break down the business processes of Spotify. Users are able to create accounts, choose subscription plans, listen to songs, and save songs to their playlist. Artists will be able to release albums and the albums will contain songs. The platform will then track the user's listening history in order to support recommendations that have been personally picked for each customer.

Now, here are a few operational transactions within Spotify. First, adding a new customer requires them to provide personal information such as their name, username, and after they provide this information, they can choose whether or not they want to purchase a subscription to Spotify.

Secondly, if a customer wants to add a song to their playlist they must select a song from an Artist's album and add it to their desired playlist. When a user wants to listen to that song from the playlist, we need to record the user, what song they are listening to, what playlist they are listening to, and the timestamp and listening time. This data is recorded across the tables User, Playlist, Playlist_Song, Song, and Listening_History. Playlist_Song is an essential entity as a playlist will have many songs, and each song can be on many different playlists.

Lastly, If an artist is releasing an album, relationships need to be recorded between the artist, songs, and album. A non-mandatory one-to-many relationship is established between the album and the songs as one album has many songs, however songs aren't necessarily on an album in the case of singles. A mandatory one-to-many relationship is also established between albums and artists as an Artist can have many albums, and an album has to have one artist.

Logical Model
Exhibit 1



Database Design

This first draft of our logical database design highlights the planned entities along with their attributes. The goal was to be able to represent all important aspects of using Spotify on the user, artist, and company end. Artists are able to create songs and albums while having defining characteristics about themselves such as genre, country, and name. Users are able to listen to songs and albums while also adding songs to playlists. The feature of adding songs to playlists is supported by the entity PlaylistSong. Finally, from the business perspective of Spotify, user information is able to be consistently logged with both subscription information and listening history as entities. This allows for Spotify to keep track of listening and subscription trends to provide a consistently unique user experience. Below, we will continue and explain the relationships between the entities in our logical model.

Database Relationships

Defining the relationships between our entities was the next step in designing our logical model. We wanted each entity to be accurately linked between relevant attributes. First, each artist has a one to many relationship with both song and album. Each artist can have many songs, while each song can only have one artist. In the same way, each artist can have many albums but

each album can only have one artist. These are linked through the primary and foreign key ArtistID.

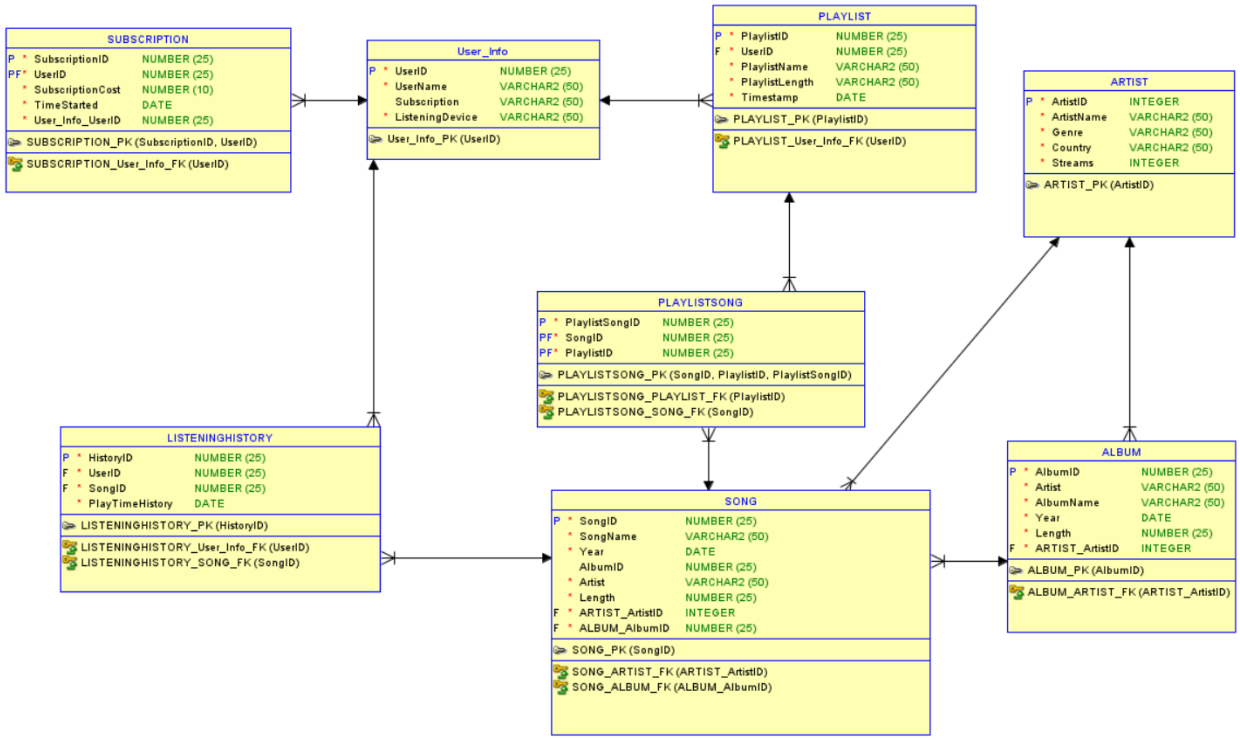
In the same way, songs and albums have a one to many relationship. One album will have many songs but a song can't be on multiple albums. This relationship is also non-mandatory since a song can be released without being on an album in the case of a single.

The next relationship is for a song to be added to a playlist. This is a slightly more complicated relationship as one song can be on many playlists as a playlist can have many songs. To achieve this we created the intermediate entity PlaylistSong to support the many to many relationship between song and playlist. PlaylistSong has two PrimaryForeign keys for both SongID and PlaylistID where it acts as the individual song entity that exists only on a single playlist.

Playlists can also be created by a single user, while a user can create many playlists, resulting in a one to many relationship between users and playlists. The user has a UserID Primary Key which acts as a foreign key for each Playlist. UserID is also a primary foreign key for the entity subscription. This is because there is a one-to-one relationship between subscription and user info as each user can only have one subscription and each subscription can only belong to a single user. It is mandatory for the subscription to have a UserId, but not for a user to have a SubscriptionID as some users have a free account.

The final entity ListeningHistory acts as an account of each time a song is listened to. It has a foreign key to connect the user who listened to the song and a foreign key to the SongID which was listened to. These are one to many relationships as a song can be listened to multiple times, and an individual can listen to multiple songs.

Physical Model
Exhibit 2



Database Normalization

We normalized our database model to Third Normal Form (3NF) in order to remove all redundancy and making sure our model is clearly structured. In our First Normal Form (1NF), we made sure that all of our attributes in our table had no repeating values. For example, instead of storing multiple songs in the Playlist table, songs are stored in a separate Playlist_Song table. The next step we decided to take was to evaluate the entities for Second Normal Form (2NF) to ensure that all non-key attributes depend on the entire primary key. This is especially important for the Playlist_Song table, where both PlaylistID and SongID are required to uniquely identify a record. To achieve Third Normal Form (3NF), the derived fields were removed from the tables. Examples of redundant attributes, such as “Songs” in Playlist, “Albums” in Artist, and “Songs” in Album, were removed because they can be derived through relationships. This ensures that the database avoids duplication and maintains consistency.

Operational SQL Queries

SUBQUERY:

```
SELECT SongName, Artist, Length
FROM Song
WHERE Song.Length > (SELECT AVG(Length) FROM Song);
```

Explanation: This query shows songs within the database that are longer than the average song length of all songs within the database. This allows us to observe which songs are longer in length than the average song and decide to further analyze patterns within other information that might reflect their songs being longer.

INNER JOIN:

```
SELECT Song.SongName, Artist.ArtistName
FROM SONG
INNER JOIN Artist ON Song.artist_artistid = Artist.ArtistID
```

Explanation: This query shows the artist's name with the song from the song table. This allows us to observe which artist wrote each song instead of just the associated artistID.

OUTER JOIN:

```
SELECT User_Info.UserName, Playlist.PlaylistName
FROM User_info
LEFT OUTER JOIN Playlist ON User_Info.UserID = Playlist.UserID
ORDER BY User_Info.UserName;
```

Explanation: This query shows all users and their playlists, including users who do not have any playlists. This allows us to observe which users tend to have more playlists than others, those that have a lot fewer, or those that have none at all.

GROUP BY:

```
SELECT SongID, COUNT(*) AS TotalPlays
FROM ListeningHistory
GROUP BY SongID
ORDER BY TotalPlays DESC;
```

Explanation: This query groups listening history by song and counts how many times each song was played. This helps Spotify identify which songs are being listened to the most.

GROUP BY & HAVING:

```
SELECT artist.artistname, SUM(song.length) AS TotalPlaytime
FROM ARTIST
JOIN SONG ON ARTIST.artistid = SONG.artist_artistid
GROUP BY artist.artistname
HAVING SUM(song.length) >= 12
ORDER BY SUM(song.length) DESC;
```

Explanation: This query combines information from song and artist tables in order to sum total play time for all of an artist's songs from song length information provided. This allows us to see which artists have a total length of all their songs summed together above 12.

INDEXES:

```
CREATE INDEX listeninghistoryuserindex ON LISTENINGHISTORY(UserID);
```

Explanation: Indexes listening history by UserID. Users listen to multiple songs across the platform so this allows for quicker retrieval by indexing them this way.

```
CREATE INDEX playlistuserindex ON PLAYLIST(UserID);
```

Explanation: Indexes playlists by UserID. Users often have multiple playlists so this allows for quicker retrieval by indexing them this way.

```
CREATE INDEX songartistindex ON SONG(artist_artistid);
```

Explanation: Indexes songs by ArtistID. Artists often have multiple songs so indexing songs this way allows for quicker retrieval.

```
CREATE INDEX albumartistindex ON ALBUM(artist_artistid);
```

Explanation: Indexes albums by ArtistID. Artists often have multiple albums or collections of songs so indexing them this way allows for quicker retrieval.

View

```
CREATE VIEW AllSongInfo AS
        SELECT song.songID, song.songName, song.Year, song.length, artist.artistname AS
ArtistName, artist.genre, album.albumID
        FROM SONG
        JOIN ARTIST ON SONG.artist_artistid = ARTIST.artistID
        JOIN ALBUM ON SONG.albumID = ALBUM.albumID;
```

Explanation: Shows all of the information about a song, including artist name and what album it is a part of. Useful as not all possible song data is available within the Song table.

PROCEDURE:

```
CREATE OR REPLACE PROCEDURE Record_Song_Play (
   Playlist_UserID NUMBER,
   Playlist_SongID NUMBER,
   Playlist_PlayTimeHistory DATE
)
AS
BEGIN
   INSERT INTO ListeningHistory
      (HistoryID, UserID, SongID, PlayTimeHistory)
   VALUES
      ((SELECT NVL(MAX(HistoryID), 0) + 1 FROM ListeningHistory),
       Playlist_UserID,
       Playlist_SongID,
       Playlist_PlayTimeHistory);

   UPDATE Artist
   SET Streams = Streams + 1
   WHERE ArtistID = (
      SELECT Artist_ArtistID
      FROM Song
      WHERE SongID = Playlist_SongID
   );
END;
/
```

Explanation: This procedure records when a user plays a song on the platform. It inserts a new record into the ListeningHistory table with the user ID, song ID, and play time, ensuring that each listening event is stored and tracked. It also updates the Artist table by increasing the stream count for the artist associated with that song. This allows the system to maintain accurate stream totals and track artist popularity. By combining both actions into one procedure, it improves efficiency and ensures consistency in how listening data is recorded and updated.

TRIGGER

```
CREATE TRIGGER LogNewArtist
AFTER INSERT ON ARTIST
FOR EACH ROW
BEGIN
        UPDATE ARTIST SET Streams = 0
        WHERE ArtistID = :new.ArtistID;

END;
/
```

Explanation: This trigger automatically sets an artist's streams to 0 if they are inserted as a new artist within the database. This allows the database to be quickly updated if a new artist is added to the artist table. When artists join Spotify, they have zero streams, so this will quickly reflect that.

Conclusion

The design and implementation of the relational database for Spotify fully addresses the demand for identifying personal trends and creating tailor-made data. Now, Spotify can use this new database to overview and coordinate the various business processes that come with identifying personal trends and creating tailor-made data. By doing this, Spotify can now keep up with the fast-growing industry and keep its edge on the competition.

Through the process of normalizing the data across eight well defined entities and enforcing referential integrity, we were able to eliminate redundancies to lead to a higher level of data quality. Examples of redundant attributes, such as “Songs” in Playlist, “Albums” in Artist, and “Songs” in Album, were removed because they can be derived through relationships. This ensures that the database avoids duplication and maintains consistency.

The operational SQL objects we developed are able to deliver immediate value in creating tailor-made data and identifying personal trends. Some examples of this are as follows:

- The Outer Join query shows all users and their playlists, including users who do not have any playlists. This allows us to observe which users tend to have more playlists than others, those that have a lot fewer, or those that have none at all. Using this information, Spotify can better understand user trends and better support users who tend to have little to few playlists.
- In the Group By and Having query, the query groups listening history by song and counts how many times each song was played. This helps Spotify identify which songs are being listened to the most. By using this information Spotify can use tailor-made data to push songs that people listen to repeatedly, rather than songs that people only listen to once and don’t listen to again. By doing this, the song recommendations by Spotify will significantly improve.
- The Procedure records when a user plays a song on the platform. It inserts a new record ensuring that each listening event is stored and tracked. It also updates the Artist table by increasing the stream count for the artist associated with that song. This allows the system to maintain accurate stream totals and track artist popularity. By using this procedure it improves efficiency and ensures consistency in how listening data is recorded and updated. This helps Spotify identify personal trends and create tailor-made data.

All of these queries and data collection included in Spotify's relational database will help with identifying personal trends, creating tailor-made data, keep up with the ever-growing music industry, and help Spotify keep an edge on their competitors. By implementing these new systems, we believe that Spotify will receive instant benefits and insights, and will be given a strong new foundation that they need in order to continue to grow their brand and excel within the industry.

Appendix

Exhibit 2 DDL

```
-- Generated by Oracle SQL Developer Data Modeler 24.3.1.347.1153
--   at:        2026-04-28 11:25:18 EDT
--   site:      Oracle Database 11g
--   type:      Oracle Database 11g



-- predefined type, no DDL - MDSYS.SDO_GEOMETRY

-- predefined type, no DDL - XMLTYPE

CREATE TABLE album (
    albumid         NUMBER(25) NOT NULL,
    artist          VARCHAR2(50) NOT NULL,
    albumname       VARCHAR2(50) NOT NULL,
    year            DATE NOT NULL,
    length          NUMBER(25) NOT NULL,
    artist_artistid INTEGER NOT NULL
);

ALTER TABLE album ADD CONSTRAINT album_pk PRIMARY KEY ( albumid );

CREATE TABLE artist (
    artistid   INTEGER NOT NULL,
    artistname VARCHAR2(50) NOT NULL,
    genre      VARCHAR2(50) NOT NULL,
    country    VARCHAR2(50) NOT NULL,
    streams    INTEGER NOT NULL
);

ALTER TABLE artist ADD CONSTRAINT artist_pk PRIMARY KEY ( artistid );

CREATE TABLE listeninghistory (
    historyid       NUMBER(25) NOT NULL,
    userid          NUMBER(25) NOT NULL,
    songid          NUMBER(25) NOT NULL,
```

```
    playtimehistory DATE NOT NULL
);

ALTER TABLE listeninghistory ADD CONSTRAINT listeninghistory_pk PRIMARY KEY (
historyid );

CREATE TABLE playlist (
    playlistid     NUMBER(25) NOT NULL,
    userid         NUMBER(25) NOT NULL,
    playlistname   VARCHAR2(50) NOT NULL,
    playlistlength VARCHAR2(50) NOT NULL,
    timestamp      DATE NOT NULL
);

ALTER TABLE playlist ADD CONSTRAINT playlist_pk PRIMARY KEY ( playlistid );

CREATE TABLE playlistsong (
    playlistsongid NUMBER(25) NOT NULL,
    songid         NUMBER(25) NOT NULL,
    playlistid     NUMBER(25) NOT NULL
);

ALTER TABLE playlistsong
    ADD CONSTRAINT playlistsong_pk PRIMARY KEY ( songid,
                                         playlistid,
                                         playlistsongid );

CREATE TABLE song (
    songid         NUMBER(25) NOT NULL,
    songname       VARCHAR2(50) NOT NULL,
    year           DATE NOT NULL,
    albumid        NUMBER(25),
    artist         VARCHAR2(50) NOT NULL,
    length         NUMBER(25) NOT NULL,
    artist_artistid INTEGER NOT NULL,
    album_albumid   NUMBER(25) NOT NULL
);

ALTER TABLE song ADD CONSTRAINT song_pk PRIMARY KEY ( songid );
```

```
CREATE TABLE subscription (
    subscriptionid   NUMBER(25) NOT NULL,
    userid           NUMBER(25) NOT NULL,
    subscriptioncost NUMBER(10) NOT NULL,
    timestarted      DATE NOT NULL,
    user_info_userid NUMBER(25) NOT NULL
);

ALTER TABLE subscription ADD CONSTRAINT subscription_pk PRIMARY KEY (
subscriptionid,
                                                    userid );

CREATE TABLE user_info (
    userid          NUMBER(25) NOT NULL,
    username        VARCHAR2(50) NOT NULL,
    subscription    VARCHAR2(50),
    listeningdevice VARCHAR2(50) NOT NULL
);

ALTER TABLE user_info ADD CONSTRAINT user_info_pk PRIMARY KEY ( userid );

ALTER TABLE album
    ADD CONSTRAINT album_artist_fk FOREIGN KEY ( artist_artistid )
        REFERENCES artist ( artistid );

ALTER TABLE listeninghistory
    ADD CONSTRAINT listeninghistory_song_fk FOREIGN KEY ( songid )
        REFERENCES song ( songid );

ALTER TABLE listeninghistory
    ADD CONSTRAINT listeninghistory_user_info_fk FOREIGN KEY ( userid )
        REFERENCES user_info ( userid );

ALTER TABLE playlist
    ADD CONSTRAINT playlist_user_info_fk FOREIGN KEY ( userid )
        REFERENCES user_info ( userid );

ALTER TABLE playlistsong
    ADD CONSTRAINT playlistsong_playlist_fk FOREIGN KEY ( playlistid )
        REFERENCES playlist ( playlistid );
```

```
ALTER TABLE playlistsong
   ADD CONSTRAINT playlistsong_song_fk FOREIGN KEY ( songid )
      REFERENCES song ( songid );

ALTER TABLE song
   ADD CONSTRAINT song_album_fk FOREIGN KEY ( album_albumid )
      REFERENCES album ( albumid );

ALTER TABLE song
   ADD CONSTRAINT song_artist_fk FOREIGN KEY ( artist_artistid )
      REFERENCES artist ( artistid );

ALTER TABLE subscription
   ADD CONSTRAINT subscription_user_info_fk FOREIGN KEY ( userid )
      REFERENCES user_info ( userid );



-- Oracle SQL Developer Data Modeler Summary Report:
--
-- CREATE TABLE                             8
-- CREATE INDEX                             0
-- ALTER TABLE                             17
-- CREATE VIEW                              0
-- ALTER VIEW                               0
-- CREATE PACKAGE                           0
-- CREATE PACKAGE BODY                      0
-- CREATE PROCEDURE                         0
-- CREATE FUNCTION                          0
-- CREATE TRIGGER                           0
-- ALTER TRIGGER                            0
-- CREATE COLLECTION TYPE                   0
-- CREATE STRUCTURED TYPE                   0
-- CREATE STRUCTURED TYPE BODY              0
-- CREATE CLUSTER                           0
-- CREATE CONTEXT                           0
-- CREATE DATABASE                          0
-- CREATE DIMENSION                         0
-- CREATE DIRECTORY                         0
```

-- CREATE DISK GROUP 0
-- CREATE ROLE 0
-- CREATE ROLLBACK SEGMENT 0
-- CREATE SEQUENCE 0
-- CREATE MATERIALIZED VIEW 0
-- CREATE MATERIALIZED VIEW LOG 0
-- CREATE SYNONYM 0
-- CREATE TABLESPACE 0
-- CREATE USER 0
--
-- DROP TABLESPACE 0
-- DROP DATABASE 0
--
-- REDACTION POLICY 0
--
-- ORDS DROP SCHEMA 0
-- ORDS ENABLE SCHEMA 0
-- ORDS ENABLE OBJECT 0
--
-- ERRORS 0
-- WARNINGS 0

Insert Statements

User info

INSERT INTO User_Info (UserName, UserID, Subscription, ListeningDevice) VALUES('Alex Johnson', 5383485, '5383485Sub', 'Mobile');
INSERT INTO User_Info (Name, User_ID, Subscription, Listening_Device) VALUES('Brian Jones', 086576, '086576Sub', 'Desktop');
INSERT INTO User_Info (UserName, UserID, Subscription, ListeningDevice) VALUES ('Bob Mcgee', 6732312, '6732312Sub', 'Tablet');
INSERT INTO User_Info (UserName, UserID, Subscription, ListeningDevice) VALUES ('Pete Dave', 987687, '987687Sub', 'Mobile');
INSERT INTO User_Info (UserName, UserID, Subscription, ListeningDevice) VALUES ('Max Millian', 5643232, '5643232Sub', 'Laptop');
INSERT INTO User_Info (UserName, UserID, Subscription, ListeningDevice) VALUES ('James Roll', 1276340, '1276340Sub', 'Desktop');

INSERT INTO User_Info (UserName, UserID, Subscription, ListeningDevice) VALUES ('Piper Morgan', 9087681, '9087681Sub', 'Laptop');
INSERT INTO User_Info (UserName, UserID, Subscription, ListeningDevice) VALUES ('Charlotte Park', 575627, '575627Sub', 'Mobile');
INSERT INTO User_Info (UserName, UserID, Subscription, ListeningDevice) VALUES ('Maddie Rose', 4539291, '4539291Sub', 'Tablet');
INSERT INTO User_Info (UserName, UserID, Subscription, ListeningDevice) VALUES ('Rosa Boyer', 2343431, '2343431Sub', 'Desktop');

Artist

INSERT INTO ARTIST (ArtistID, ArtistName, Genre, Country, Streams) VALUES (1, 'Michael Jackson', 'Pop', 'USA', 2000000);
INSERT INTO ARTIST (ArtistID, ArtistName, Genre, Country, Streams) VALUES (2, 'AC/DC', 'Rock', 'Australia', 1800000);
INSERT INTO ARTIST (ArtistID, ArtistName, Genre, Country, Streams) VALUES (3, 'Pink Floyd', 'Rock', 'UK', 1500000);
INSERT INTO ARTIST (ArtistID, ArtistName, Genre, Country, Streams) VALUES (4, 'Fleetwood Mac', 'Rock', 'UK', 1400000);
INSERT INTO ARTIST (ArtistID, ArtistName, Genre, Country, Streams) VALUES (5, 'The Beatles', 'Rock', 'UK', 2500000);
INSERT INTO ARTIST (ArtistID, ArtistName, Genre, Country, Streams) VALUES (6, 'Adele', 'Pop', 'UK', 1700000);
INSERT INTO ARTIST (ArtistID, ArtistName, Genre, Country, Streams) VALUES (7, 'Taylor Swift', 'Pop', 'USA', 2200000);
INSERT INTO ARTIST (ArtistID, ArtistName, Genre, Country, Streams) VALUES (8, 'Kendrick Lamar', 'Hip-Hop', 'USA', 1600000);
INSERT INTO ARTIST (ArtistID, ArtistName, Genre, Country, Streams) VALUES (9, 'Dua Lipa', 'Pop', 'UK', 1300000);
INSERT INTO ARTIST (ArtistID, ArtistName, Genre, Country, Streams) VALUES (10, 'Frank Ocean', 'RnB', 'USA', 900000);

Album

INSERT INTO album (albumid, artist, AlbumName, year, length, artist_artistid) VALUES (20, 'Michael Jackson', 'Thriller', TO_DATE('1982-01-01','YYYY-MM-DD'), 42, 1);
INSERT INTO album (albumid, artist, AlbumName, year, length, artist_artistid) VALUES (21, 'AC/DC', 'Back in Black', TO_DATE('1980-01-01','YYYY-MM-DD'), 42, 2);

```
INSERT INTO album (albumid, artist, AlbumName, year, length, artist_artistid)
VALUES (22, 'Pink Floyd', 'The Dark Side of the Moon',
TO_DATE('1973-01-01','YYYY-MM-DD'), 43, 3);
INSERT INTO album (albumid, artist, AlbumName, year, length, artist_artistid)
VALUES (23, 'Fleetwood Mac', 'Rumours', TO_DATE('1977-01-01','YYYY-MM-DD'), 39,
4);
INSERT INTO album (albumid, artist, AlbumName, year, length, artist_artistid)
VALUES (24, 'The Beatles', 'Abbey Road', TO_DATE('1969-01-01','YYYY-MM-DD'), 47,
5);
INSERT INTO album (albumid, artist, AlbumName, year, length, artist_artistid)
VALUES (25, 'Adele', '21', TO_DATE('2011-01-01','YYYY-MM-DD'), 48, 6);
INSERT INTO album (albumid, artist, AlbumName, year, length, artist_artistid)
VALUES (26, 'Taylor Swift', '1989', TO_DATE('2014-01-01','YYYY-MM-DD'), 49, 7);
INSERT INTO album (albumid, artist, AlbumName, year, length, artist_artistid)
VALUES (27, 'Kendrick Lamar', 'DAMN.', TO_DATE('2017-01-01','YYYY-MM-DD'), 55,
8);
INSERT INTO album (albumid, artist, AlbumName, year, length, artist_artistid)
VALUES (28, 'Dua Lipa', 'Future Nostalgia', TO_DATE('2020-01-01','YYYY-MM-DD'),
37, 9);
INSERT INTO album (albumid, artist, AlbumName, year, length, artist_artistid)
VALUES (29, 'Frank Ocean', 'Blonde', TO_DATE('2016-01-01','YYYY-MM-DD'), 60, 10);
```

Song

```
INSERT INTO SONG (SongID, SongName, Year, AlbumID, Artist, Length,
ARTIST_ArtistID, ALBUM_AlbumID)
VALUES (101, 'Billie Jean', TO_DATE('1982-01-01','YYYY-MM-DD'), 20, 'Michael
Jackson', 294, 1, 20);
INSERT INTO SONG (SongID, SongName, Year, AlbumID, Artist, Length,
ARTIST_ArtistID, ALBUM_AlbumID)
VALUES (102, 'Hells Bells', TO_DATE('1980-01-01','YYYY-MM-DD'), 21, 'AC/DC', 312,
2, 21);
INSERT INTO SONG (SongID, SongName, Year, AlbumID, Artist, Length,
ARTIST_ArtistID, ALBUM_AlbumID)
VALUES (103, 'Time', TO_DATE('1973-01-01','YYYY-MM-DD'), 22, 'Pink Floyd', 413, 3,
22);
INSERT INTO SONG (SongID, SongName, Year, AlbumID, Artist, Length,
ARTIST_ArtistID, ALBUM_AlbumID)
```

VALUES (104, 'Dreams', TO_DATE('1977-01-01','YYYY-MM-DD'), 23, 'Fleetwood Mac', 257, 4, 23);
INSERT INTO SONG (SongID, SongName, Year, AlbumID, Artist, Length, ARTIST_ArtistID, ALBUM_AlbumID)
VALUES (105, 'Come Together', TO_DATE('1969-01-01','YYYY-MM-DD'), 24, 'The Beatles', 259, 5, 24);
INSERT INTO SONG (SongID, SongName, Year, AlbumID, Artist, Length, ARTIST_ArtistID, ALBUM_AlbumID)
VALUES (106, 'Rolling in the Deep', TO_DATE('2011-01-01','YYYY-MM-DD'), 25, 'Adele', 228, 6, 25);
INSERT INTO SONG (SongID, SongName, Year, AlbumID, Artist, Length, ARTIST_ArtistID, ALBUM_AlbumID)
VALUES (107, 'Blank Space', TO_DATE('2014-01-01','YYYY-MM-DD'), 26, 'Taylor Swift', 231, 7, 26);
INSERT INTO SONG (SongID, SongName, Year, AlbumID, Artist, Length, ARTIST_ArtistID, ALBUM_AlbumID)
VALUES (108, 'HUMBLE.', TO_DATE('2017-01-01','YYYY-MM-DD'), 27, 'Kendrick Lamar', 177, 8, 27);
INSERT INTO SONG (SongID, SongName, Year, AlbumID, Artist, Length, ARTIST_ArtistID, ALBUM_AlbumID)
VALUES (109, 'Levitating', TO_DATE('2020-01-01','YYYY-MM-DD'), 28, 'Dua Lipa', 203, 9, 28);

Playlist

INSERT INTO PLAYLIST (PlaylistID, UserID, PlaylistName, PlaylistLength, Timestamp)
VALUES (1, 5383485, 'Workout Mix', 4200, TO_DATE('2007-05-12','YYYY-MM-DD'));
INSERT INTO PLAYLIST (PlaylistID, UserID, PlaylistName, PlaylistLength, Timestamp)
VALUES (2, 086576, 'Chill Hits', 3600, TO_DATE('2020-03-24','YYYY-MM-DD'));
INSERT INTO PLAYLIST (PlaylistID, UserID, PlaylistName, PlaylistLength, Timestamp)
VALUES (3, 6732312, 'Classic Rock', 4000, TO_DATE('2002-10-31','YYYY-MM-DD'));
INSERT INTO PLAYLIST (PlaylistID, UserID, PlaylistName, PlaylistLength, Timestamp)
VALUES (4, 987687, 'Focus Mode', 3800, TO_DATE('2005-11-25','YYYY-MM-DD'));
INSERT INTO PLAYLIST (PlaylistID, UserID, PlaylistName, PlaylistLength, Timestamp)
VALUES (5, 5643232, 'Late Night', 3500, TO_DATE('2024-02-24','YYYY-MM-DD'));
INSERT INTO PLAYLIST (PlaylistID, UserID, PlaylistName, PlaylistLength, Timestamp)
VALUES (6, 1276340, 'Pop Essentials', 3900, TO_DATE('2019-05-12','YYYY-MM-DD'));
INSERT INTO PLAYLIST (PlaylistID, UserID, PlaylistName, PlaylistLength, Timestamp)

```
VALUES (7, 9087681, 'Hip-Hop Vibes', 4100, TO_DATE('2016-10-26','YYYY-MM-DD'));
INSERT INTO PLAYLIST (PlaylistID, UserID, PlaylistName, PlaylistLength, Timestamp)
VALUES (8, 575627, 'Indie Chill', 3700, TO_DATE('2011-08-23','YYYY-MM-DD'));
INSERT INTO PLAYLIST (PlaylistID, UserID, PlaylistName, PlaylistLength, Timestamp)
VALUES (9, 4539291, 'Throwbacks', 4300, TO_DATE('2000-09-17','YYYY-MM-DD'));
INSERT INTO PLAYLIST (PlaylistID, UserID, PlaylistName, PlaylistLength, Timestamp)
VALUES (10, 2343431, 'Top Tracks', 4000, TO_DATE('2023-07-04','YYYY-MM-DD'));
```

PlaylistSong

```
INSERT INTO PLAYLISTSONG (PlaylistSongID, SongID, PlaylistID)
VALUES (1, 101, 1);
INSERT INTO PLAYLISTSONG (PlaylistSongID, SongID, PlaylistID)
VALUES (2, 104, 1);
INSERT INTO PLAYLISTSONG (PlaylistSongID, SongID, PlaylistID)
VALUES (3, 102, 2);
INSERT INTO PLAYLISTSONG (PlaylistSongID, SongID, PlaylistID)
VALUES (4, 109, 2);
INSERT INTO PLAYLISTSONG (PlaylistSongID, SongID, PlaylistID)
VALUES (5, 103, 3);
INSERT INTO PLAYLISTSONG (PlaylistSongID, SongID, PlaylistID)
VALUES (6, 104, 3);
INSERT INTO PLAYLISTSONG (PlaylistSongID, SongID, PlaylistID)
VALUES (7, 106, 4);
INSERT INTO PLAYLISTSONG (PlaylistSongID, SongID, PlaylistID)
VALUES (8, 108, 4);
INSERT INTO PLAYLISTSONG (PlaylistSongID, SongID, PlaylistID)
VALUES (9, 105, 5);
INSERT INTO PLAYLISTSONG (PlaylistSongID, SongID, PlaylistID)
VALUES (10, 103, 5);
```

Subscription

```
INSERT INTO SUBSCRIPTION (SUBSCRIPTIONID, USERID, SUBSCRIPTIONCOST,
TIMESTARTED, USER_INFO_USERID)
VALUES (1, 5383485, 12, TO_DATE('2015-10-12','YYYY-MM-DD'), 5383485);
INSERT INTO SUBSCRIPTION (SUBSCRIPTIONID, USERID, SUBSCRIPTIONCOST,
TIMESTARTED, USER_INFO_USERID)
VALUES (2, 86756, 12, TO_DATE('2016-09-12','YYYY-MM-DD'), 86756);
```

```
INSERT INTO SUBSCRIPTION (SUBSCRIPTIONID, USERID, SUBSCRIPTIONCOST,
TIMESTARTED, USER_INFO_USERID)
VALUES (3, 6732312, 16, TO_DATE('2017-08-12','YYYY-MM-DD'), 6732312);
INSERT INTO SUBSCRIPTION (SUBSCRIPTIONID, USERID, SUBSCRIPTIONCOST,
TIMESTARTED, USER_INFO_USERID)
VALUES (4, 987687, 12, TO_DATE('2018-07-12','YYYY-MM-DD'), 987687);
INSERT INTO SUBSCRIPTION (SUBSCRIPTIONID, USERID, SUBSCRIPTIONCOST,
TIMESTARTED, USER_INFO_USERID)
VALUES (5, 5643232, 16, TO_DATE('2019-06-12','YYYY-MM-DD'), 5643232);
INSERT INTO SUBSCRIPTION (SUBSCRIPTIONID, USERID, SUBSCRIPTIONCOST,
TIMESTARTED, USER_INFO_USERID)
VALUES (6, 1276340, 16, TO_DATE('2020-05-12','YYYY-MM-DD'), 1276340);
INSERT INTO SUBSCRIPTION (SUBSCRIPTIONID, USERID, SUBSCRIPTIONCOST,
TIMESTARTED, USER_INFO_USERID)
VALUES (7, 9087681, 12, TO_DATE('2021-04-12','YYYY-MM-DD'), 9087681);
INSERT INTO SUBSCRIPTION (SUBSCRIPTIONID, USERID, SUBSCRIPTIONCOST,
TIMESTARTED, USER_INFO_USERID)
VALUES (8, 575627, 12, TO_DATE('2022-03-12','YYYY-MM-DD'), 575627);
INSERT INTO SUBSCRIPTION (SUBSCRIPTIONID, USERID, SUBSCRIPTIONCOST,
TIMESTARTED, USER_INFO_USERID)
VALUES (9, 4539291, 12, TO_DATE('2023-02-12','YYYY-MM-DD'), 4539291);
INSERT INTO SUBSCRIPTION (SUBSCRIPTIONID, USERID, SUBSCRIPTIONCOST,
TIMESTARTED, USER_INFO_USERID)
VALUES (10, 2343431, 16, TO_DATE('2024-01-12','YYYY-MM-DD'), 2343431);
```

Listening History

```
INSERT INTO LISTENINGHISTORY (HISTORYID, USERID, SONGID,
PLAYTIMEHISTORY)
VALUES (1, 5383485, 103, TO_DATE('2026-04-29','YYYY-MM-DD'));
INSERT INTO LISTENINGHISTORY (HISTORYID, USERID, SONGID,
PLAYTIMEHISTORY)
VALUES (2, 86756, 102, TO_DATE('2026-04-29','YYYY-MM-DD'));
INSERT INTO LISTENINGHISTORY (HISTORYID, USERID, SONGID,
PLAYTIMEHISTORY)
VALUES (3, 6732312, 109, TO_DATE('2026-04-29','YYYY-MM-DD'));
```

```
INSERT INTO LISTENINGHISTORY (HISTORYID, USERID, SONGID,
PLAYTIMEHISTORY)
VALUES (4, 987687, 107, TO_DATE('2026-04-29','YYYY-MM-DD'));
INSERT INTO LISTENINGHISTORY (HISTORYID, USERID, SONGID,
PLAYTIMEHISTORY)
VALUES (5, 5643232, 106, TO_DATE('2026-04-29','YYYY-MM-DD'));
INSERT INTO LISTENINGHISTORY (HISTORYID, USERID, SONGID,
PLAYTIMEHISTORY)
VALUES (6, 1276340, 107, TO_DATE('2026-04-29','YYYY-MM-DD'));
INSERT INTO LISTENINGHISTORY (HISTORYID, USERID, SONGID,
PLAYTIMEHISTORY)
VALUES (7, 9087681, 101, TO_DATE('2026-04-29','YYYY-MM-DD'));
INSERT INTO LISTENINGHISTORY (HISTORYID, USERID, SONGID,
PLAYTIMEHISTORY)
VALUES (8, 575627, 108, TO_DATE('2026-04-29','YYYY-MM-DD'));
INSERT INTO LISTENINGHISTORY (HISTORYID, USERID, SONGID,
PLAYTIMEHISTORY)
VALUES (9, 4539291, 104, TO_DATE('2026-04-29','YYYY-MM-DD'));
INSERT INTO LISTENINGHISTORY (HISTORYID, USERID, SONGID,
PLAYTIMEHISTORY)
VALUES (10, 2343431, 105, TO_DATE('2026-04-29','YYYY-MM-DD'));
```

Link to all sql
https://drive.google.com/file/d/1EWpShgJcNSCfN5yboQ0zYjdXuC_93Oee/view?usp=sharing